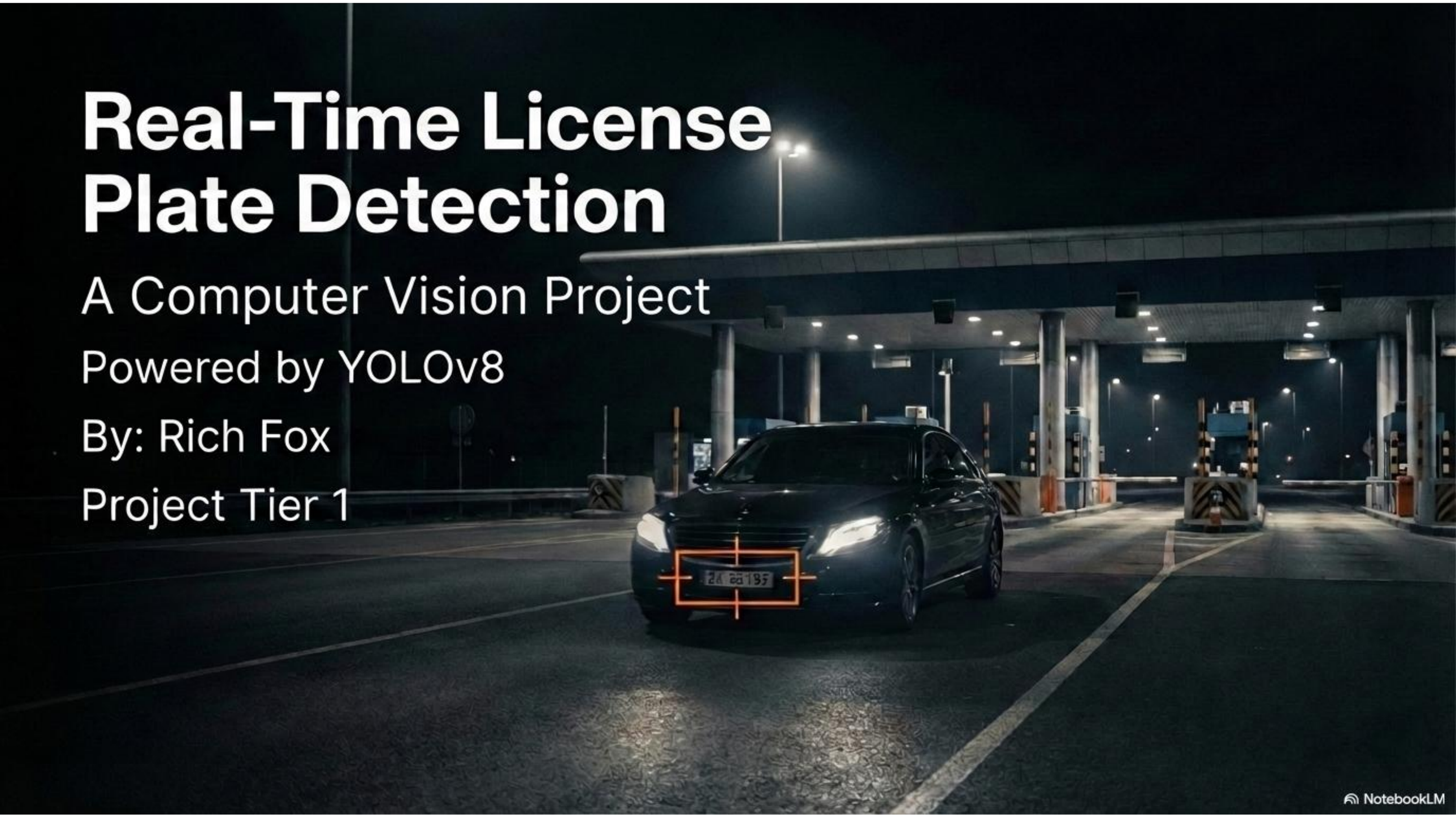
Real-Time License Plate Detection

A Computer Vision Project

Powered by YOLOv8

By: Rich Fox

Project Tier 1



NotebookLM

The Problem: Manual Vehicle Monitoring is Slow, Expensive, and Error-Prone

Three core areas demanding automated solutions:

1. Traffic Monitoring:
Who cares: Urban planners, commuters.
Why it's important: Manual oversight cannot scale with urban growth, leading to congestion and inefficiency.

2. Toll Collection:
Who cares: Toll operators, drivers.
Why it's important: Physical stopping at toll booths creates significant delays, increases emissions, and reduces throughput.

3. Security:
Who cares: Security personnel, property owners.
Why it's important: Retrospective manual video review for unauthorized vehicles is slow, error-prone, and ineffective for real-time threat response.

NotebookLM

Automated, instantaneous vehicle recognition at the edge

We are building an automated pipeline that detects and reads license plates in **real-time video feeds** using lightweight object detection.

NotebookLM

Precision at speed with YOLOv8 Nano

Built on the Ultralytics framework, utilizing YOLOv8n for three critical advantages:

- **Lightweight Architecture:** The smallest variant (nano) guarantees real-time FPS even on low-low-end edge GPUs.
- **Anchor-Free Design:** Dramatically improves recognition accuracy on small, angled objects like license plates.
- **Deployment Ready:** Strikes the perfect balance between memory footprint and detection precision compared to older V5/V7 models.

NotebookLM

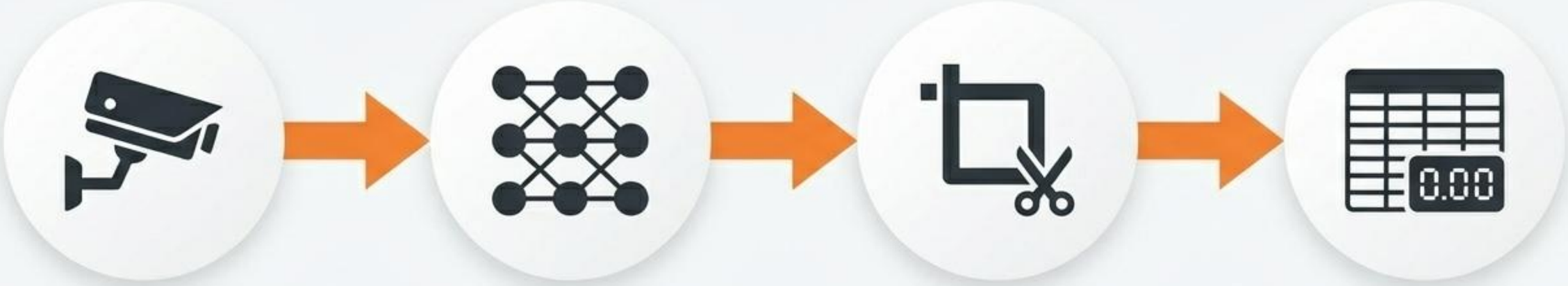
Training on diverse, real-world conditions

Sourcing robust training data via Kaggle:

- **Primary Data:** Automatic Number Plate Recognition dataset.
- **Video Validation:** License Plate Dataset (Car number plate video).
- **Pipeline Management:** Strict Train/Val/Test splits managed via **YOLO data.yaml** configuration.

NotebookLM

The real-time detection pipeline



Input: Raw streaming video or high-resolution imagery.

Processing: YOLOv8n isolates the bounding box of the vehicle and plate.

Extraction: System crops the localized region of interest.

Output: High-confidence detection metric and isolated plate imagery.

NotebookLM

Defining measurable success

90%+

Primary Metric: Detection Accuracy across varied angles and lighting.

Real-Time FPS

Secondary Metric: Processing speed capable of handling live video without frame drops.

NotebookLM

Execution timeline from data to deployment

Week 10: March 27: Acquire datasets, configure environment, establish data.yaml.

Week 11: April 3: Train and fine-tune YOLOv8n model.

Week 12: April 10: Evaluate metrics, test against video inputs.

Week 13: April 17: Develop real-time video processing demo.

Week 14: April 24: Finalize testing and GitHub repository documentation.

Week 15: May 1: Live presentation and deployment.

NotebookLM

Proactive risk mitigation

Hurdle: Insufficient detection accuracy on skewed angles.

Failsafe: Implement Albumentations library for aggressive data augmentation (rotations, blur, lighting shifts).

Hurdle: YOLOv8n struggles with complex edge cases.

Failsafe: Scale up architecture to YOLOv8s (small) or integrate supplemental Roboflow public datasets.

NotebookLM

Ready to build today

Compute Engine: Kaggle & Google Colab Notebooks.
Core Frameworks: Ultralytics YOLO & PyTorch.

Total Cost: $0

NotebookLM